

Hyperparameter Tuning ANN using Optuna - Complete Notes

Table of Contents

1. [What is Hyperparameter Tuning?](#)
 2. [Why Optuna?](#)
 3. [Hyperparameters We Will Tune](#)
 4. [Plan of Action](#)
 5. [Optuna High-Level Overview](#)
 6. [Complete Code Implementation](#)
 7. [Experiment Tracking with MLflow](#)
 8. [Tips for Better Results](#)
-

What is Hyperparameter Tuning?

In-Depth Explanation

When building a neural network, we make many decisions:

- How many hidden layers?
- How many neurons per hidden layer?
- What learning rate to use?
- What batch size?
- How many epochs?
- What dropout rate?
- Which optimizer?
- What regularization strength?

In the previous videos, the instructor made these decisions based purely on **intuition** (2 hidden layers, 128 and 64 neurons, learning rate 0.01, etc.). However, in real-world projects, you cannot rely on intuition alone. You need a systematic way to find the best combination of these values. This process is called **Hyperparameter Tuning**.

Why can't we just use our intuition?

- Different datasets require different architectures
- What works for one problem may fail for another
- The optimal values are often counter-intuitive
- Real-world projects demand evidence-based decisions

What is Optuna?

Optuna is a powerful **hyperparameter optimization framework** that uses **Bayesian Search** (specifically Tree-structured Parzen Estimator - TPE). Unlike Grid Search or Random Search, Bayesian Search:

- Learns from past trials
 - Intelligently chooses which values to try next
 - Finds optimal parameters in fewer trials
 - Can handle complex search spaces with dependencies
-

Why Optuna?

Method	How it works	Pros	Cons
Grid Search	Tries all combinations	Exhaustive	Very slow
Random Search	Random combinations	Faster than Grid	No learning
Bayesian Search (Optuna)	Learns from past trials	Efficient, intelligent	More complex

Optuna Advantages:

1. Automatically prunes bad trials early
 2. Supports dynamic search spaces
 3. Built-in visualization
 4. Can handle conditional hyperparameters
 5. Very popular in industry
-

Hyperparameters We Will Tune

We will tune **8 hyperparameters** in total:

#	Hyperparameter	Current Value	Search Range
1	Number of Hidden Layers	2	1 to 5
2	Neurons per Layer	128, 64	8 to 128 (step 8)
3	Epochs	100	10 to 50 (step 10)
4	Learning Rate	0.01	1e-5 to 1e-1 (log scale)
5	Dropout Rate	0.3	0.1 to 0.5
6	Batch Size	32	16, 32, 64, 128
7	Optimizer	SGD	Adam, SGD, RMSprop

#	Hyperparameter	Current Value	Search Range
8	Weight Decay (L2 Regularization)	0	1e-5 to 1e-3 (log scale)

Plan of Action

We will run Optuna in **two rounds**:

Round 1 (Learning Round)

- **Goal:** Learn how to use Optuna with PyTorch
- **Tune only:** Number of hidden layers + Neurons per layer
- **Keep others constant**
- **Trials:** 10

Round 2 (Full Tuning Round)

- **Goal:** Find optimal values for all 8 hyperparameters
- **Tune:** All 8 hyperparameters
- **Trials:** 10+ (can increase to 50 for better results)

Optuna High-Level Overview

The Objective Function Pattern

```
def objective(trial):
    # 1. Define search space (suggest hyperparameters)
    hidden_layers = trial.suggest_int('n_hidden_layers', 1, 5)
    neurons = trial.suggest_int('neurons_per_layer', 8, 128, step=8)

    # 2. Create model with these hyperparameters
    model = MyNeuralNetwork(input_dim, output_dim, hidden_layers,
                             neurons)

    # 3. Define loss function and optimizer
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=learning_rate)

    # 4. Training loop
    for epoch in range(epochs):
        train_one_epoch()

    # 5. Evaluate and return accuracy
    accuracy = evaluate()
    return accuracy
```

Study Object Pattern

```
# Create a study
study = optuna.create_study(direction='maximize') # maximize accuracy

# Run optimization
study.optimize(objective, n_trials=10)

# Get best results
best_accuracy = study.best_value
best_params = study.best_params
```

Complete Code Implementation

Step 1: Import Libraries and Setup GPU

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision
import torchvision.transforms as transforms
import numpy as np
import optuna

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

Step 2: Load and Prepare Data (Same as before)

```
# Load Fashion MNIST dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=True, download=True,
    transform=transforms.ToTensor()
)
test_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=False, download=True,
    transform=transforms.ToTensor()
)

# Normalize the data
mean = train_dataset.data.float().mean() / 255.0
std = train_dataset.data.float().std() / 255.0

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[mean], std=[std])
])
```

```

])

# Re-apply with normalization
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=False, download=True, transform=transform
)

# Create data loaders (batch size will be tuned, so we set later)
# We'll create loaders inside the objective function

```

Step 3: Dynamic Neural Network Class

This is the most important part. The network architecture must change based on hyperparameters.

```

class MyNeuralNetwork(nn.Module):
    def __init__(self, input_dim, output_dim, num_hidden_layers,
        neurons_per_layer, dropout_rate):
        super(MyNeuralNetwork, self).__init__()

        # Create a list to store all layers dynamically
        layers = []

        # Input layer to first hidden layer
        current_input = input_dim

        # Loop to create hidden layers
        for i in range(num_hidden_layers):
            # Add Linear layer
            layers.append(nn.Linear(current_input, neurons_per_layer))

            # Add Batch Normalization
            layers.append(nn.BatchNorm1d(neurons_per_layer))

            # Add Activation Function (ReLU)
            layers.append(nn.ReLU())

            # Add Dropout
            layers.append(nn.Dropout(dropout_rate))

            # Update input dimension for next layer
            current_input = neurons_per_layer

        # Output layer
        layers.append(nn.Linear(current_input, output_dim))

```

```

    # Create sequential model
    self.model = nn.Sequential(*layers)

def forward(self, x):
    # Flatten the input (28x28 images to 784 vector)
    x = x.view(x.size(0), -1)
    return self.model(x)

```

How the dynamic architecture works:

Let's say we have:

- num_hidden_layers = 2
- neurons_per_layer = 128
- input_dim = 784
- output_dim = 10

The layers list will contain:

```

1. Linear(784, 128)
2. BatchNorm1d(128)
3. ReLU()
4. Dropout(0.3)
5. Linear(128, 128)      # Second hidden layer
6. BatchNorm1d(128)
7. ReLU()
8. Dropout(0.3)
9. Linear(128, 10)      # Output layer

```

Step 4: Round 1 - Simple Objective Function (Only Layers + Neurons)

```

def objective_round1(trial):
    # ===== STEP 1: SUGGEST HYPERPARAMETERS =====
    # Suggest number of hidden layers (1 to 5)
    num_hidden_layers = trial.suggest_int('num_hidden_layers', 1, 5)

    # Suggest neurons per layer (8 to 128, step 8)
    neurons_per_layer = trial.suggest_int('neurons_per_layer', 8, 128,
step=8)

    # Fixed hyperparameters for Round 1
    epochs = 50
    learning_rate = 0.01
    batch_size = 32
    dropout_rate = 0.3
    weight_decay = 0

    # ===== STEP 2: CREATE DATA LOADERS =====
    train_loader = DataLoader(train_dataset, batch_size=batch_size,

```

```

shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size,
shuffle=False)

# ===== STEP 3: CREATE MODEL =====
input_dim = 784 # 28x28 images flattened
output_dim = 10 # 10 classes

model = MyNeuralNetwork(
    input_dim, output_dim,
    num_hidden_layers, neurons_per_layer,
    dropout_rate
).to(device)

# ===== STEP 4: LOSS AND OPTIMIZER =====
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=learning_rate,
weight_decay=weight_decay)

# ===== STEP 5: TRAINING LOOP =====
for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        # Forward pass
        output = model(data)
        loss = criterion(output, target)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

# ===== STEP 6: EVALUATION =====
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        _, predicted = torch.max(output.data, 1)
        total += target.size(0)

```

```

        correct += (predicted == target).sum().item()

    accuracy = 100 * correct / total

    # Return accuracy (Optuna will try to maximize this)
    return accuracy

```

Step 5: Run Round 1 Optimization

```

# Install optuna if not already installed
# !pip install optuna

import optuna

# Create study (direction='maximize' because we want highest accuracy)
study_round1 = optuna.create_study(direction='maximize')

# Run optimization for 10 trials
study_round1.optimize(objective_round1, n_trials=10)

# View results
print("\n===== ROUND 1 RESULTS =====")
print(f"Best accuracy: {study_round1.best_value:.2f}%")
print(f"Best parameters: {study_round1.best_params}")

```

Expected Output (similar to):

```

===== ROUND 1 RESULTS =====
Best accuracy: 89.32%
Best parameters: {'num_hidden_layers': 3, 'neurons_per_layer': 120}

```

Step 6: Round 2 - Full Objective Function (All 8 Parameters)

```

def objective_round2(trial):
    # ===== STEP 1: SUGGEST ALL 8 HYPERPARAMETERS =====

    # 1. Number of hidden layers
    num_hidden_layers = trial.suggest_int('num_hidden_layers', 1, 5)

    # 2. Neurons per layer
    neurons_per_layer = trial.suggest_int('neurons_per_layer', 8, 128,
step=8)

    # 3. Epochs (10 to 50, step 10)
    epochs = trial.suggest_int('epochs', 10, 50, step=10)

    # 4. Learning Rate (log scale from 1e-5 to 1e-1)
    learning_rate = trial.suggest_float('learning_rate', 1e-5, 1e-1,
log=True)

```



```

# 5. Dropout Rate (0.1 to 0.5)
dropout_rate = trial.suggest_float('dropout_rate', 0.1, 0.5,
step=0.1)

# 6. Batch Size (categorical choices)
batch_size = trial.suggest_categorical('batch_size', [16, 32, 64,
128])

# 7. Optimizer Name (categorical)
optimizer_name = trial.suggest_categorical('optimizer', ['Adam',
'SGD', 'RMSprop'])

# 8. Weight Decay (log scale from 1e-5 to 1e-3)
weight_decay = trial.suggest_float('weight_decay', 1e-5, 1e-3,
log=True)

# ===== STEP 2: CREATE DATA LOADERS WITH TUNED BATCH SIZE
=====
train_loader = DataLoader(train_dataset, batch_size=batch_size,
shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
shuffle=False)

# ===== STEP 3: CREATE MODEL =====
input_dim = 784
output_dim = 10

model = MyNeuralNetwork(
    input_dim, output_dim,
    num_hidden_layers, neurons_per_layer,
    dropout_rate
).to(device)

# ===== STEP 4: LOSS FUNCTION =====
criterion = nn.CrossEntropyLoss()

# ===== STEP 5: OPTIMIZER (conditional on optimizer_name)
=====
if optimizer_name == 'Adam':
    optimizer = optim.Adam(model.parameters(), lr=learning_rate,
weight_decay=weight_decay)
elif optimizer_name == 'SGD':
    optimizer = optim.SGD(model.parameters(), lr=learning_rate,
weight_decay=weight_decay)
else: # RMSprop
    optimizer = optim.RMSprop(model.parameters(),
lr=learning_rate, weight_decay=weight_decay)

```

```

# ===== STEP 6: TRAINING LOOP =====
for epoch in range(epochs):
    model.train()
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)

        # Forward pass
        output = model(data)
        loss = criterion(output, target)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# ===== STEP 7: EVALUATION =====
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        _, predicted = torch.max(output.data, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()

accuracy = 100 * correct / total
return accuracy

```

Step 7: Run Round 2 Optimization

```

# Create study for Round 2
study_round2 = optuna.create_study(direction='maximize')

# Run optimization (can increase n_trials for better results)
study_round2.optimize(objective_round2, n_trials=10)

# View results
print("\n===== ROUND 2 RESULTS =====")
print(f"Best accuracy: {study_round2.best_value:.2f}%")
print(f"Best parameters:")
for key, value in study_round2.best_params.items():
    print(f"  {key}: {value}")

```

Expected Output:

```
===== ROUND 2 RESULTS =====
Best accuracy: 89.73%
Best parameters:
  num_hidden_layers: 4
  neurons_per_layer: 88
  epochs: 20
  learning_rate: 0.0025
  dropout_rate: 0.2
  batch_size: 128
  optimizer: RMSprop
  weight_decay: 0.00012
```

Step 8: Visualize Optuna Results

```
# Plot optimization history
optuna.visualization.plot_optimization_history(study_round2)

# Plot parameter importances
optuna.visualization.plot_param_importances(study_round2)

# Plot parallel coordinate plot
optuna.visualization.plot_parallel_coordinate(study_round2)

# Plot contour plot
optuna.visualization.plot_contour(study_round2)
```

Experiment Tracking with MLflow

What is Experiment Tracking?

When you run multiple hyperparameter tuning trials, you should track:

- Which parameters were used in each trial
- What accuracy was achieved
- Visualize relationships between parameters and accuracy

MLflow Dashboard Output (as shown in video)

After running 50 trials with MLflow, the instructor showed:

Parameter	Best Value	Insight
Number of Hidden Layers	2 or 3	Most trials concentrated here
Neurons per Layer	100-120	Highest density in this range
Epochs	40-50	Best results around 40

Parameter	Best Value	Insight
Learning Rate	0.002-0.004	Sweet spot in this range
Optimizer	SGD	Gave best accuracy
Dropout Rate	0.2	Most consistent results

Key Insight from Scatter Plots:

- More trials were run on certain parameter values (Optuna learns)
- The parameters with most trials are generally the best ones
- You can see clear relationships between params and accuracy

Tips for Better Results

1. Increase Number of Trials

```
# Instead of 10 trials, try 50 or 100
study.optimize(objective, n_trials=50)
```

2. Expand Search Ranges

```
# Wider range for hidden layers
num_hidden_layers = trial.suggest_int('num_hidden_layers', 1, 10)

# Wider range for neurons
neurons_per_layer = trial.suggest_int('neurons_per_layer', 16, 256,
step=16)

# More epochs
epochs = trial.suggest_int('epochs', 10, 100, step=10)
```

3. Add More Optimizers

```
optimizer_name = trial.suggest_categorical('optimizer',
['Adam', 'SGD', 'RMSprop', 'AdamW', 'Nadam']
)
```

4. Use Pruning for Early Stopping

```
# Add pruning to stop bad trials early
for epoch in range(epochs):
    # ... training code ...

    # Report intermediate value
    trial.report(accuracy, epoch)

    # Handle pruning
    if trial.should_prune():
        raise optuna.TrialPruned()
```

5. Use Different Direction for Loss

```
# For loss (minimize)
study = optuna.create_study(direction='minimize')

# For accuracy (maximize)
study = optuna.create_study(direction='maximize')
```

Complete Summary

What We Learned:

1. **Why Hyperparameter Tuning is necessary** - Intuition is not enough for real-world projects
2. **Optuna basics** - Bayesian search is more efficient than grid/random search
3. **Dynamic neural network architecture** - Code that adapts to different numbers of layers
4. **Objective function pattern** - Suggest → Create → Train → Evaluate → Return
5. **Study optimization** - Create study → Run trials → Get best parameters
6. **Experiment tracking** - MLflow helps visualize relationships

Key Code Structure to Remember:

```
# 1. Define dynamic model class
class DynamicNN(nn.Module):
    def __init__(self, input_dim, output_dim, n_layers, n_neurons,
dropout):
        # Build layers dynamically using a loop
        # Store in nn.Sequential

# 2. Define objective function
def objective(trial):
    # Suggest parameters
    param = trial.suggest_int('name', low, high)

    # Create model with suggested params
    # Train and evaluate
    # Return metric to optimize

# 3. Run optimization
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

# 4. Get best results
```

```
best_params = study.best_params
best_value = study.best_value
```

Common Pitfalls to Avoid:

Pitfall	Solution
Too few trials	Use at least 30-50 trials
Too narrow search space	Expand ranges initially
Forgetting .to(device)	Move model and data to GPU
Not flattening input	Use x.view(x.size(0), -1)
Wrong direction	Maximize accuracy, minimize loss

Practice Exercise

Try to tune these additional hyperparameters:

1. **Activation functions** (ReLU, LeakyReLU, ELU)
2. **Number of CNN layers** (if using CNN instead of ANN)
3. **Kernel sizes** (for CNN)
4. **Learning rate schedulers**

Final Notes

- This same approach works for **CNN, RNN, Transformers** - any PyTorch model
- Optuna is industry-standard (used at Google, Microsoft, etc.)
- Always combine hyperparameter tuning with experiment tracking
- Start with small number of trials, then increase once code works

Remember: The goal is not just to get the highest accuracy, but to understand how different hyperparameters affect your model's performance on your specific dataset!

Building a CNN using PyTorch - Complete Notes

Video Source: CampusX Video 11